

베이지안 기계학습

3. Markov chain Monte Carlo (MCMC) II

이경재

Table of Contents

- ▶ Hamiltonian Monte Carlo (HMC)
- ▶ Installing rstan
- ▶ Exercise using rstan
- ▶ Remark and next week

Hamiltonian Monte Carlo (HMC)

Hamiltonian Monte Carlo: motivation

- ▶ Random walk Metropolis has been commonly used.
- ▶ Unfortunately, the performance of random walk scales poorly with increasing dimension and complexity of the target distribution.

Hamiltonian Monte Carlo: motivation

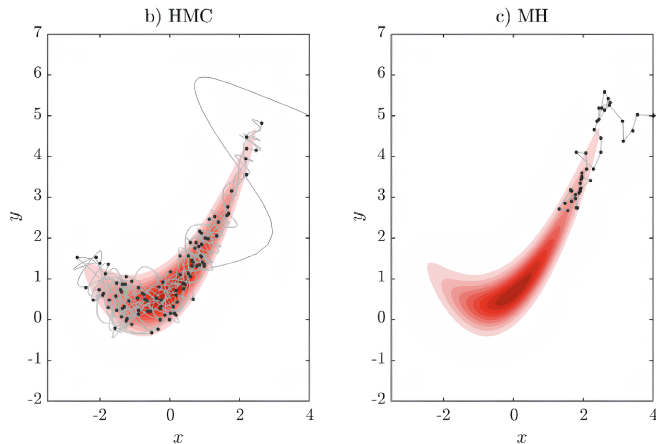


Figure: Source: Wang et al. (2019) Hamiltonian Monte Carlo methods for Subset Simulation in reliability analysis

Hamiltonian Monte Carlo

- ▶ **Hamiltonian Monte Carlo (HMC)** borrows an idea from physics to overcome the local random walk behavior in the Metropolis algorithm.
- ▶ It allows the sampler to explore much more efficiently the target distribution.
- ▶ Use **Hamiltonian dynamics** and a latent “momentum” vector, so the steps within an iteration move along a “trajectory”.

*Hamiltonian dynamics

- ▶ Hamiltonian dynamics:

$$H(q, p) = U(q) + K(p),$$

where $U(q)$ the potential energy and $K(p)$ is the kinetic energy.

- ▶ $q \in \mathbb{R}^d$ is the position, and $p \in \mathbb{R}^d$ is the momentum.

*Hamiltonian dynamics

$$H(q, p) = U(q) + K(p),$$

- ▶ In fact, we will consider $q = \theta$ and $U(q) = -\log\{\pi(\theta)f(x | \theta)\}$.
- ▶ $K(p)$ is usually defined as

$$K(p) = \frac{1}{2}p^T M^{-1}p,$$

where $M \in \mathbb{R}^{d \times d}$ is a diagonal matrix.

- ▶ It means that $p \sim N_d(0, M)$.

*HMC: proposal

- ▶ Let $f(q, p)$ be the target density, where

$$f(q, p) = \frac{1}{Z} \exp \{ -H(q, p) \}.$$

- ▶ We will use Hamiltonian dynamics as a Metropolis kernel.
- ▶ Specifically, we will sample (calculate) (q, p) from the current position (q_0, p_0) among

$$\left\{ (q, p) : H(q, p) = H(q_0, p_0) \right\},$$

which preserves the Hamiltonian $H(q, p)$.

*HMC: proposal

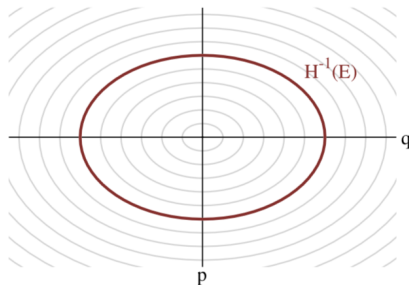


Figure: from Betancourt (2017) A conceptual introduction to Hamiltonian Monte Carlo.

- ▶ The proposal using the Hamilton's equations eventually cannot escape from the orbit, which is not very desirable.

*HMC: proposal

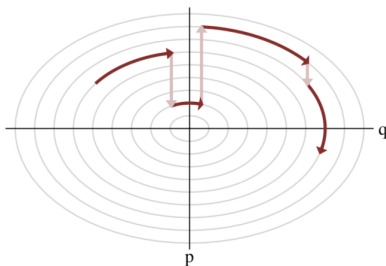


Figure: from Betancourt (2017) A conceptual introduction to Hamiltonian Monte Carlo.

- ▶ The problem will be addressed by suggesting a new momentum p at the start of each iteration.
- ▶ Then, the proposed approach allows for the exploration of the entire space of q .

HMC: remark

- ▶ Unfortunately, in most cases, we cannot solve the Hamilton's equations in closed form.
- ▶ An numerical integration should be used (leapfrog method).
- ▶ For more detail (if you want), see
 - Betancourt (2017) *A conceptual introduction to Hamiltonian Monte Carlo* [[Link](#)] or
 - Neal (2012) *MCMC using Hamiltonian dynamics* [[Link](#)].

HMC: remark

- ▶ Studying all of these things to perform HMC can be quite challenging. Is there an easier way?
- ▶ `rstan` will do it for you!

next

HMC: remark

- ▶ Studying all of these things to perform HMC can be quite challenging. Is there an easier way?
- ▶ **rstan** will do it for you!

next

Table of Contents

- ▶ Hamiltonian Monte Carlo (HMC)
- ▶ **Installing rstan**
- ▶ Exercise using rstan
- ▶ Remark and next week

Installing rstan

What is Stan?

- ▶ [Stan](#) is a program developed in 2012 primarily by the team led by Andrew Gelman and Bob Carpenter for statistical inference.
- ▶ The name “Stan” is derived from Stanislaw Ulam, one of the developers of the Monte Carlo method.
- ▶ Stan is built on C++ and is compatible with various programming languages such as Python, R, Julia, etc. It can operate on different environments, including Windows, Mac and Linux.

What we can do using Stan

- ▶ Using Stan, you can perform the following tasks:
 - Bayesian inference using Markov Chain Monte Carlo (MCMC), employing Hamiltonian Monte Carlo (HMC) methods.
 - Bayesian inference using variational methods.
 - Penalized maximum likelihood estimation using optimization techniques.

Install: 1. Windows

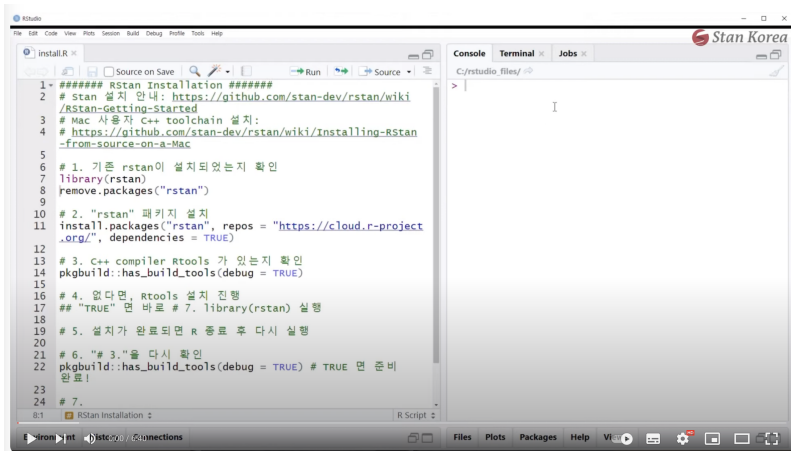
1. Download Rtools: `https:`

`//cran.r-project.org/bin/windows/Rtools/`

2. Open R and enter the following command:

```
install.packages("rstan", dependencies = TRUE)
```

Install: 1. Windows



The screenshot shows the RStudio interface with a script file named 'install.R'. The script contains the following R code for installing RStan on Windows:

```
1 ##### RStan Installation #####
2 # stan 설치 안내: https://github.com/stan-dev/rstan/wiki
  /RStan-Getting-Started
3 # Mac 사용자 C++ toolchain 설치:
4 # https://github.com/stan-dev/rstan/wiki/Installing-RStan
  -from-source-on-a-Mac
5
6 # 1. 기존 rstan이 설치되었는지 확인
7 library(rstan)
8 remove.packages("rstan")
9
10 # 2. "rstan" 패키지 설치
11 install.packages("rstan", repos = "https://cloud.r-project
  .org/", dependencies = TRUE)
12
13 # 3. C++ compiler Rtools 가 있는지 확인
14 pkgbuild::has_build_tools(debug = TRUE)
15
16 # 4. 없다면, Rtools 설치 진행
17 ## "TRUE" 면 바로 # 7. library(rstan) 실행
18
19 # 5. 설치가 완료되면 R 종료 후 다시 실행
20
21 # 6. "# 3."을 다시 확인
22 pkgbuild::has_build_tools(debug = TRUE) # TRUE 면 준비
  완료!
23
24 # 7.
```

[스탠코리아] (Windows) R에서 Stan 시작하기 | RStan 설치 튜토리얼 | Stan으로 배우는 베이지 통계학



StanKorea 스탠코리아

구독자 1.1만명

구독

👍 27



🔗 공유

📄 오프라인 저장



🔖 저장



Figure: 스탠코리아 유튜브 영상

(https://www.youtube.com/watch?v=_viWdItrwGc).

Install: 2. Mac

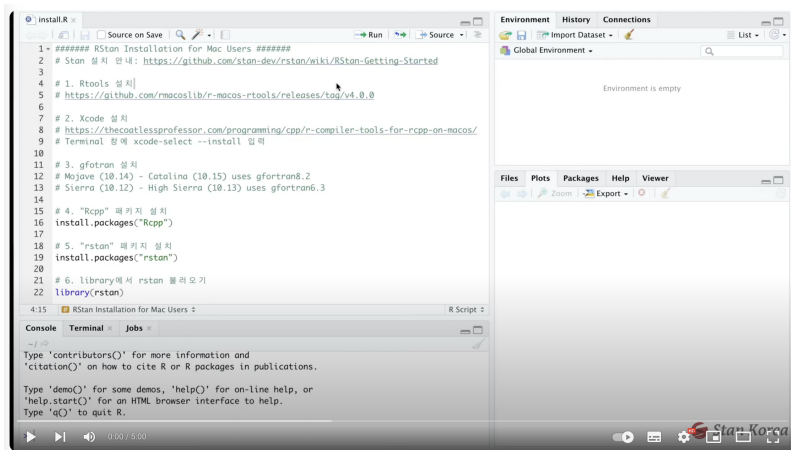
1. Install Xcode: Download from Mac App Store
2. In the terminal, run the following command:

```
xcode-select --install
```

3. Open R and enter the following command:

```
install.packages("rstan", dependencies = TRUE)
```

Install: 2. Mac



[스탠코리아] (Mac) R에서 Stan 시작하기 | RStan 설치 튜토리얼 | Stan으로 배우는 베이지 통계학



StanKorea 스탠코리아
구독자 1.1만명



11



Figure: 스탠코리아 유튜브 영상

(<https://www.youtube.com/watch?v=YC6fFfkG41Q>).

More information

- ▶ If you visit the [RStan Korean GitHub wiki](#), you can find detailed instructions for installing RStan:

`https://github.com/stan-dev/rstan/wiki/
RStan-Getting-Started-(%ED%95%9C%EA%B5%AD%EC%
96%B4)`

- ▶ Or you can visit [StanKorea 스탠코리아](#):

`https://www.youtube.com/@stankorea313.`

- ▶ And here is the official website: `http://mc-stan.org.`

Table of Contents

- ▶ Hamiltonian Monte Carlo (HMC)
- ▶ Installing rstan
- ▶ **Exercise using rstan**
- ▶ Remark and next week

Exercise using rstan

Stan code

Name of code block	Purpose
Data	External input
Transformed data	Data pre-processing
Parameters	Guidelines for sampling space determination
Transformed parameters	Parameter pre-processing
Model	Guidelines for building posterior
Generated quantities	Post-processing

Figure: Types of code blocks in Stan from [Hilbe, de Souza and Ishida (2017) Bayesian Models for Astrophysical Data. Cambridge University Press].

- Stan code consists of six blocks.

Stan code: basic syntax

- Usually, we can use these three blocks: data, parameters and model:

```
data{  
  - input data -  
}
```

```
parameters{  
  - parameters to infer -  
}
```

```
model{  
  - prior distribution -  
  - likelihood function -  
}
```

Example: binomial model

$$\begin{aligned} Y_1, \dots, Y_n \mid \theta &\stackrel{iid}{\sim} B(\theta), \\ \theta &\sim \text{Beta}(1, 1) \end{aligned}$$

- data generation with $n = 1000$ and $\theta = 0.3$:

```
n <- 1000
```

```
y <- rbinom(n, 1, 0.3)
```

```
library(rstan)
```

```
library(bayesplot) # to use mcmc_acf function
```

Example: binomial model

► Stan code

```
bern.model <- "  
  data{  
    - input data -  
  
  }  
  parameters{  
    - parameters to infer -  
  }  
  model{  
    - prior distribution -  
    - likelihood function -  
  
  }  
"
```

Example: binomial model

► Stan code

```
bern.model <- "  
  data{  
    int<lower=0> n;  
    int y[n];  
  }  
  parameters{  
    - parameters to infer -  
  }  
  model{  
    - prior distribution -  
    - likelihood function -  
  
  }  
"
```

Example: binomial model

► Stan code

```
bern.model <- "  
  data{  
    int<lower=0> n;  
    int y[n];  
  }  
  parameters{  
    real<lower=0,upper=1> theta;  
  }  
  model{  
    - prior distribution -  
    - likelihood function -  
  
  }  
"
```

Example: binomial model

- Stan code

```
bern.model <- "  
  data{  
    int<lower=0> n;  
    int y[n];  
  }  
  parameters{  
    real<lower=0,upper=1> theta;  
  }  
  model{  
    theta ~ beta(1,1);  
    for(i in 1:n){  
      y[i] ~ bernoulli(theta);  
    }  
  }  
"
```


Example: binomial model

- Posterior inference using

```
stanC <- stan_model(model_code = bern.model)
stanD <- list(y = y, n = n)
stanF <- sampling(object = stanC, data = stanD,
                  chain = 3, iter = 1000,
                  warmup = 200, thin = 1)
```

Example: binomial model

- Posterior inference using

```
stanC <- stan_model(model_code = bern.model)
stanD <- list(y = y, n = n)
stanF <- sampling(object = stanC, data = stanD,
                  chain = 3, iter = 1000,
                  warmup = 200, thin = 1)
```

or equivalently,

```
stanD <- list(y = y, n = n)
stanF <- stan(model_code=bern.model, data=stanD,
              chains=3, iter=1000,
              warmup = 200, thin = 1)
```

Example: binomial model

- Summary statistics

```
print(stanF)
```

```
> print(stanF)
Inference for Stan model: anon_model.
3 chains, each with iter=1000; warmup=200; thin=1;
post-warmup draws per chain=800, total post-warmup draws=2400.
```

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
theta	0.33	0.00	0.01	0.30	0.31	0.33	0.33	0.35	1086	1
lp__	-632.59	0.02	0.65	-634.52	-632.76	-632.33	-632.15	-632.10	1047	1

```
Samples were drawn using NUTS(diag_e) at Tue Jan 16 13:42:51 2024.
For each parameter, n_eff is a crude measure of effective sample size,
and Rhat is the potential scale reduction factor on split chains (at
convergence, Rhat=1).
```

Example: binomial model

- Summary statistics (for each chain)

```
summary(stanF)
```

```
> summary(stanF)
$summary
      mean      se_mean      sd      2.5%      25%      50%      75%      97.5%    n_eff    Rhat
theta    0.3252758 0.0004444195 0.01464326  0.2974173  0.3149526  0.3253537  0.3348587  0.3540422 1085.649 0.9991804
lp__    -632.5876741 0.0202461176 0.65499923 -634.5240019 -632.7629121 -632.3297392 -632.1506573 -632.0983827 1046.642 1.0011949

$c_summary
, , chains = chain:1

      stats
parameter  mean      sd      2.5%      25%      50%      75%      97.5%
theta      0.3251783 0.01508893  0.2952853  0.3147394  0.3253139  0.3348545  0.3566167
lp__      -632.6178877 0.69779327 -634.7192393 -632.7842919 -632.3340421 -632.1504871 -632.0984551

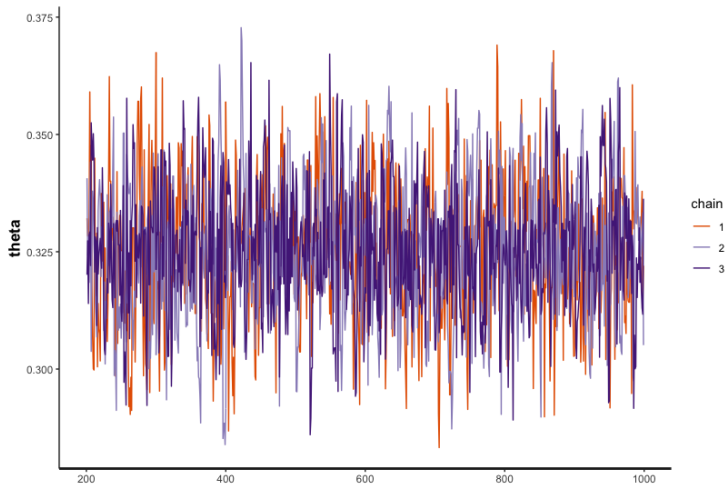
, , chains = chain:2

      stats
parameter  mean      sd      2.5%      25%      50%      75%      97.5%
theta      0.3249815 0.01483166  0.2979135  0.3140621  0.3250001  0.3349881  0.3537167
lp__      -632.6006380 0.67379581 -634.5283418 -632.7631091 -632.3483732 -632.1584431 -632.0985119
```

Example: binomial model

- ▶ Trace plot

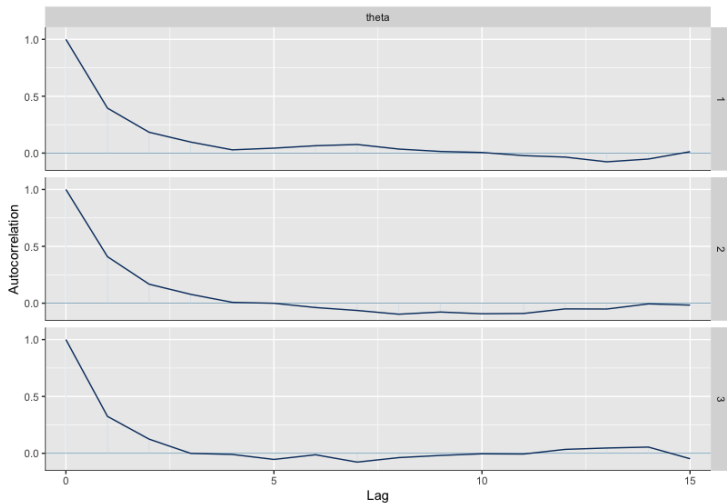
```
stan_trace(stanF)
```



Example: binomial model

- Autocorrelation plots

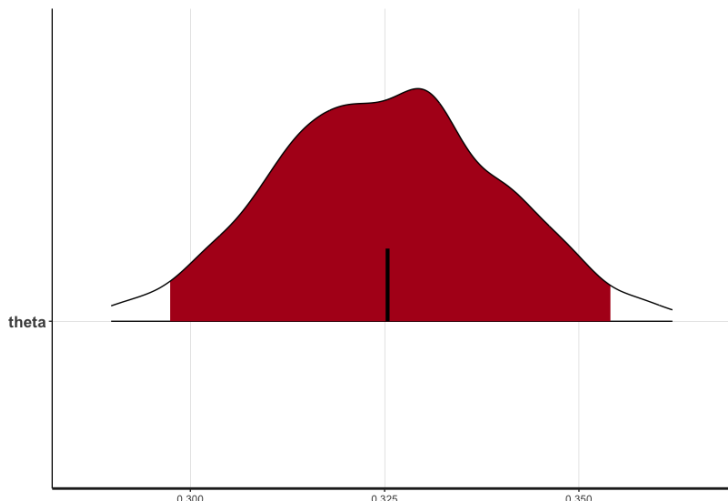
```
mcmc_acf(stanF, pars = "theta", lags = 15)
```



Example: binomial model

- Density function

```
plot(stanF, show_density = T, ci_level = 0.95,  
     outer_level = 0.99)
```



Example: normal model

$$\begin{aligned} Y_1, \dots, Y_n \mid \theta &\stackrel{iid}{\sim} N(\theta, 1), \\ \theta &\sim N(0, 100) \end{aligned}$$

- data generation with $n = 100$ and $\theta = 5$:

```
n <- 100
```

```
y <- rnorm(n, mean = 5, sd = 1)
```

```
library(rstan)
```

```
library(bayesplot) # to use mcmc_acf function
```


Example: normal model

- Stan code

```
norm.model <- "  
  data{  
    int<lower=0> n;  
    real y[n];  
  }  
  parameters{  
    real theta;  
  }  
  model{  
    for(i in 1:n){  
      y[i] ~ normal(theta, 1);  
    }  
    theta ~ normal(0, 100);  
  }  
"
```

Example: normal model

- Posterior inference

```
stanC <- stan_model(model_code = norm.model)
stanD <- list(y = y, n = n)
stanF <- sampling(object = stanC, data = stanD,
                  chain = 3, iter = 1000,
                  warmup = 200, thin = 1)
```

Example: normal model

```
print(stanF)
```

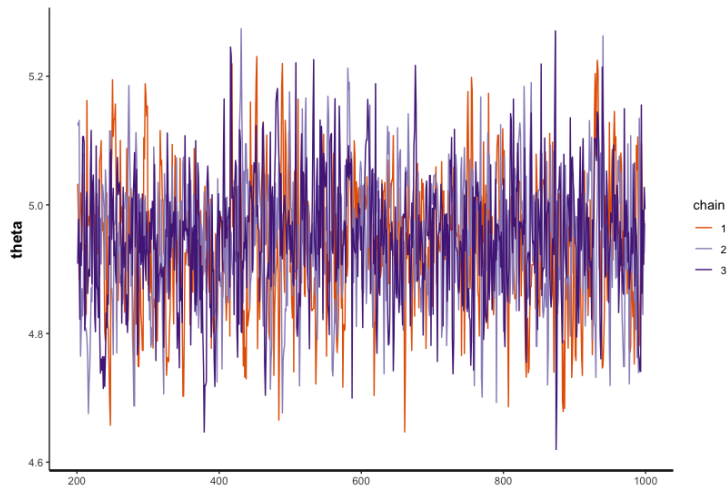
```
> print(stanF)
Inference for Stan model: anon_model.
3 chains, each with iter=1000; warmup=200; thin=1;
post-warmup draws per chain=800, total post-warmup draws=2400.
```

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
theta	4.95	0.00	0.10	4.74	4.88	4.95	5.01	5.14	876	1
lp__	-43.62	0.02	0.73	-45.76	-43.78	-43.33	-43.16	-43.11	982	1

Samples were drawn using NUTS(diag_e) at Tue Jan 16 14:57:28 2024.
For each parameter, n_eff is a crude measure of effective sample size,
and Rhat is the potential scale reduction factor on split chains (at
convergence, Rhat=1).

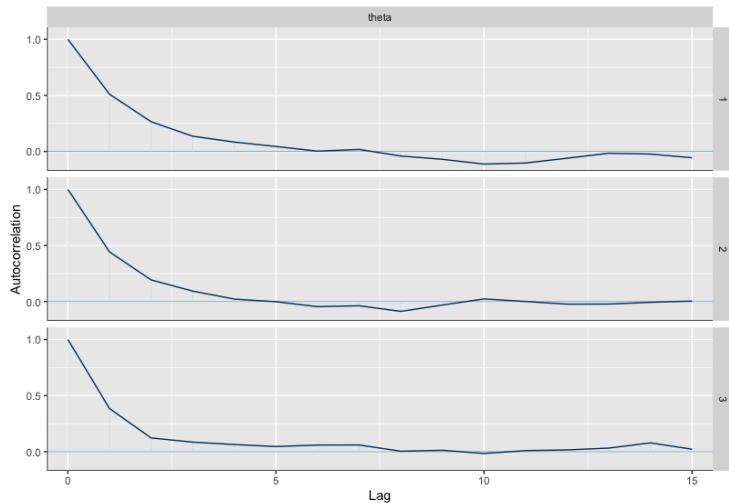
Example: normal model

```
stan_trace(stanF)
```



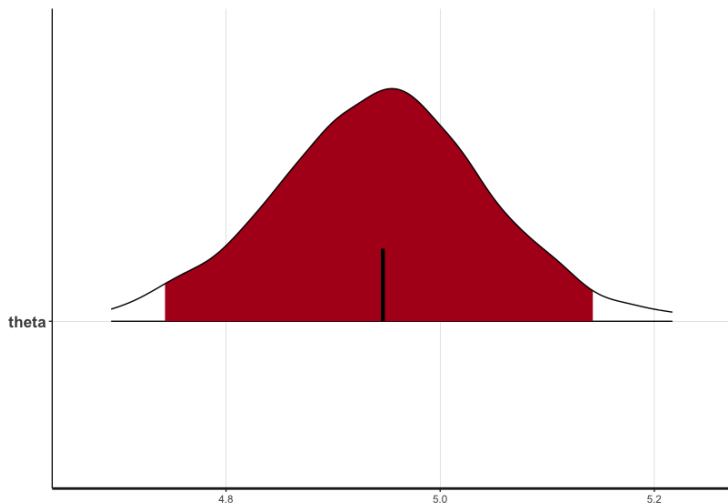
Example: normal model

```
mcmc_acf(stanF, pars = "theta", lags = 15)
```



Example: normal model

```
plot(stanF, show_density = T, ci_level = 0.95,  
     outer_level = 0.99)
```



Remark and next week

Remark

- ▶ Today, we studied the introduction and basic usage of the `rstan` package.
- ▶ This package is a very convenient tool that relies on a few basic syntax elements to automatically perform Bayesian inference.
- ▶ There are also R packages like `rjags` and `nimble` that can be used for similar purposes.

Next week

Bayesian inference for regression models I